COMPUTER SCIENCE + COMPUTER ENGINEERING FOR GAMES

09

VOLUME NINE

PACKAGE 10 / GRADUATE SPECIALIZATION

Compilers, shader languages and engine specialization

Standalone teaching module; use the workbook and separate answer key

MODULE 45 / FOUR LESSONS

Edition: September 28, 2026

Original lesson sequence preserved. Self-study materials and runnable educational references; not an accredited degree or completed learner submission.

Contents

Module 45 — Compilers, shader languages and engine specialization 3

Lesson 45.1 — Intermediate representations, SSA and dataflow analysis 4

Lesson 45.2 — Alias analysis, vectorization and legal optimization 6

Lesson 45.3 — Domain-specific languages, specialization and safe JIT execution 8

Lesson 45.4 — Compiler/hardware co-design and graduate artifact review 10

Module 45 project: compiler mini-project plus Gate I 12

Primary reading directory 13

Module 45 — Compilers, shader languages and engine specialization

Prerequisites: Modules 15-16, 18, 37 and 41-42.

Original project brief: Gate I: deliver a research-quality reproduction and extension in rendering, simulation, GPU algorithms or compilation. This module also requires a small compiler optimization with semantic tests, generated-code inspection and end-to-end measurements.

Core reference IDs: R14, R11, R37, R48. Full entries are in the reading directory.

- 45.1 — Intermediate representations, SSA and dataflow analysis
- 45.2 — Alias analysis, vectorization and legal optimization
- 45.3 — Domain-specific languages, specialization and safe JIT execution
- 45.4 — Compiler/hardware co-design and graduate artifact review

Lesson 45.1 — Intermediate representations, SSA and dataflow analysis

Learning outcome: Represent program meaning so transformations can be justified.

Planning: 24-38 hours. Includes assigned study and practical work; module/Gate I time overlaps.

Original teaching primer

An intermediate representation makes control flow, values and operations explicit. Static single assignment gives each value one defining assignment; merge points use a representation such as phi nodes to select values from predecessor paths. SSA simplifies many analyses but does not by itself establish program correctness. Dataflow analysis computes properties over a control-flow graph, often through iterative fixed points on a chosen domain. Liveness identifies values that may be used later; constant propagation tracks values known under the analysis. Soundness means the analysis does not exclude behavior relevant to its guarantee, while precision controls how much useful information it retains. Dominance and control-flow edges must remain consistent after transformations.

SSA is an explicit representation of alternatives

In static single assignment form, each value name has one definition. A control-flow merge selects a value from a predecessor through a phi node. The choice follows the edge actually taken; it is not an eager evaluation of both branches. For a loop, a header phi can choose the initial value on entry and the updated value on the backedge. Dominance requires a definition to be available on every path to an ordinary use; phi inputs have edge-specific rules. [S13]

Changing control-flow edges requires updating phi predecessors and preserving dominance. A transformation that deletes a branch but leaves an obsolete phi input is not a harmless formatting issue. In a loop with multiple phis, all incoming values are selected from the predecessor environment together; assigning them sequentially can accidentally make one phi read another's new value rather than the old edge value.

Dataflow is a fixed-point computation over a chosen abstraction

For backward liveness, OUT[B] is the union of IN sets for successor blocks and IN[B]=USE[B] union (OUT[B] minus DEF[B]). Iterate until no set changes. On a finite variable universe, a monotone formulation starting from empty sets reaches a fixed point. This computes conservative possible future uses, not the exact concrete values of every execution.

In SSA, phi operands are uses on specific incoming edges. A generic block-level liveness routine without phi-aware edge treatment needs preprocessing or extension. The included live_variables function uses an explicit ordinary USE/DEF/successor input contract; do not silently feed it phi semantics it does not implement. Work-list scheduling changes the number of iterations, not the intended least fixed point of this monotone framework.

An interpreter is a differential-testing anchor

Define integer width, overflow, branch interpretation, unsupported operations and the step limit of the reference interpreter. The supplied CFG executes unsigned 32-bit arithmetic modulo 2^32 and has a fuel limit so a malformed or nonterminating student program does not run forever. It handles branches, loop phis and bounded execution, but does not statically prove unique definitions, full dominance or type consistency for every possible IR.

Before optimizing, add a verifier for the subset you accept. Then compare original and transformed programs over directed boundary values and randomized inputs. A concrete differential test can find a bug; it does not replace a legality argument for all inputs. Preserve both the IR before/after and a minimal failing input when the comparison disagrees.

Worked engineering example

Consider an if/else that yields a=1 on the then edge and b=2 on the else edge. The merge is x=phi(then:a, else:b). With a runtime condition, the abstract constant result is not known to be 1. If constant propagation proves the condition true, it may remove the else edge and replace the phi by a, after repairing the CFG. For liveness, if a later block uses x, x is live into that block; the corresponding incoming operand is live only on the edge that supplies it in a phi-aware analysis.

Original worked check

After an if/else assigning x=1 or x=2, a merge can represent x=phi(1 from then, 2 from else). Replacing it with 1 unconditionally would be wrong unless the else path is proven unreachable.

Practical assignment

Extend the tiny compiler with a control-flow IR and one dataflow analysis. Pass: derive transfer functions and a fixed-point stopping condition, test loops and merges, and verify transformed programs against an interpreter.

Full execution steps and acceptance checks are in LAB_WORKBOOK.pdf under this same lesson ID. Your warm-up file is `code/exercises/lesson_45_1.py`.

Reference starting point: code/gradlab/compiler.py: run_cfg, live_variables

Assigned study

Original assignment: R14: LLVM frontend, control flow and optimization tutorials; R48: compiler/architecture interaction.

Focused expansion: R14 and S13: LLVM control-flow and SSA semantics; use the original tiny compiler from Module 15 as the frontend where available. The supplied CFG interpreter is an executable semantic oracle, not a complete IR verifier.

Check your understanding

Attempt all three original questions before opening the key. A lesson also requires its reading, lab and graduate problems.

1. What does SSA require of each value?
2. Why is the example phi node not always constant 1?
3. What is the role of a fixed point in dataflow analysis?

Lesson 45.2 — Alias analysis, vectorization and legal optimization

Learning outcome: Separate transformations that are fast from transformations that preserve meaning.

Planning: 22-34 hours. Includes assigned study and practical work; module/Gate I time overlaps.

Original teaching primer

An optimization must preserve the language's permitted observable behavior under its assumptions. Alias analysis determines which references may identify overlapping storage. Incorrectly assuming no aliasing can make loop reordering or vectorization wrong. Undefined behavior, overflow rules and floating-point options affect what transformations are legal, so record the compilation contract. Vectorization groups iterations only when dependencies and semantics permit it. Reductions, exceptional values and rounding can require special handling. Register allocation maps values to a limited physical register set; spills introduce memory traffic. Inspect generated code and test adversarial inputs. A faster compiler output that silently changes required results is a correctness failure, not an optimization.

Legality comes before profitability

A transformation is legal only if it preserves the permitted observable behavior under its declared language contract. For unsigned 32-bit arithmetic, additions and multiplications wrap modulo 2^32. Algebraic identities valid in that ring can justify constant folding. Signed overflow, floating-point reassociation and exception behavior obey different rules; do not reuse a proof across those contracts without checking the assumptions.

The reference native example checks (x+0)*(2+3) against x*5* for 100,000 unsigned inputs and has a simple algebraic justification. Those tests support implementation confidence; the ring identity supplies the all-input argument. Converting the example to a floating-point expression with NaN, infinities or signed zero can invalidate identities that seemed obvious over real numbers. [S13]

Alias and dependence facts constrain the schedule

Two syntactically different references can denote overlapping storage. A write through one reference may change what a later read through another observes. Before reordering, distinguish true data dependence, anti-dependence and output dependence. A loop-carried recurrence such as a[i]=a[i-1]+1 cannot simply be evaluated as independent lanes.

For half-open logical ranges [a,a+n) and [b,b+m), overlap exists when a<b+m and b<a+n, provided both lengths are positive. The reference models integer offsets. Native pointer ordering/provenance and overflow-safe address arithmetic require additional language-specific care. A runtime range check can dispatch to a vectorized no-overlap path and a conservative fallback, but a false no-alias annotation does not make overlapping inputs legal. [S15]

Inspect the complete generated computation

Request optimization remarks and inspect generated assembly or intermediate code for the actual target. An optimization may add a runtime alias guard, scalar remainder loop, gathers, conversions or spills. Count the effect on the calling workload, not just one attractive vector instruction. Short loops may spend more time dispatching than computing.

Keep semantic tests active in optimized builds. The native reference uses explicit checks rather than assertions that disappear under common Release settings. Run sanitizers separately from timing; instrumentation changes execution cost. Sweep aligned, unaligned, overlapping where permitted, empty and remainder sizes. Reject any speed result that violates the required numerical or memory contract.

Worked engineering example

Let a initially be [7,0,0,0] and apply a[i]=a[i-1]+1 for i=1,2,3 in order. The result is [7,8,9,10]. A naive simultaneous implementation that reads all original previous elements gives [7,8,1,1]. There may be a valid alternative algorithm for this particular recurrence, but proving and implementing that algorithm is different from asserting that the original iterations are independent. The adversarial initial values make the hidden dependence obvious.

Original worked check

For a[i]=a[i-1]+1, iterations depend on previous results. Treating all iterations as independent changes behavior unless a valid alternative algorithm preserves the recurrence.

Practical assignment

Implement or study a loop transformation with an explicit legality check. Pass: overlapping-memory and boundary cases are tested, compiler flags are recorded and speed claims exclude transformations that violate the required numerical contract.

Full execution steps and acceptance checks are in LAB_WORKBOOK.pdf under this same lesson ID. Your warm-up file is `code/exercises/lesson_45_2.py`.

Reference starting point: code/gradlab/compiler.py: ranges_overlap, fold; code/native/main.cpp

Assigned study

Original assignment: R14: optimization and code generation; R11: aliasing, lifetime and arithmetic guidance; R37: vector/parallel execution consequences.

Focused expansion: S13: LLVM language semantics; S15: LLVM loop and SLP vectorizers; R11: lifetime, arithmetic and aliasing contracts. Record actual compiler versions and flags, not an assumed universal optimizer behavior.

Check your understanding

Attempt all three original questions before opening the key. A lesson also requires its reading, lab and graduate problems.

1. Why is alias analysis important for reordering loads and stores?
2. Are iterations of a[i]=a[i-1]+1 independent?
3. Why record floating-point compiler options?

Lesson 45.3 — Domain-specific languages, specialization and safe JIT execution

Learning outcome: Trade generality for predictable game-engine computation.

Planning: 22-34 hours. Includes assigned study and practical work; module/Gate I time overlaps.

Original teaching primer

A domain-specific language restricts expression to make authoring, validation or optimization easier. A material graph, particle formula or animation expression can expose structure that a general compiler cannot easily infer. Define types, allowed effects and resource limits before implementing code generation. Specialization precomputes decisions known from material or scene parameters, potentially reducing runtime branches at the cost of more variants and compilation work. Just-in-time compilation introduces startup latency, code-cache management and execution-security concerns. Cache keys must include meaningful inputs and tool versions. Untrusted scripts or graphs need capability and complexity limits. A restricted interpreter remains a useful fallback and semantic reference.

Restrict the language to make its contract testable

A useful game-engine DSL may express a material formula, particle update or animation parameter rather than an arbitrary program. Define its types, effects, evaluation order and resource limits. The supplied expression language accepts unsigned 32-bit constants, named inputs, addition and multiplication. It forbids filesystem, network, calls, dynamic imports and arbitrary evaluation. Trees are validated before interpretation or code generation.

Node-count and depth limits bound the work of this tree interpreter. Inputs are range-checked. Generated input names come from a controlled identifier mapping rather than pasting user strings directly into C++ source. These controls reduce the accepted language's risk and simplify optimization, but they are not an operating-system isolation boundary. Untrusted native code still needs a separate security design.

Specialization trades runtime work for variants

A constant material option can remove a runtime branch or fold a subexpression. With k independent binary options, there can be 2^k combinations before considering formats, lighting modes or targets. Not every combination needs eager compilation. Use a bounded cache, compilation budget, prioritized common variants and a reference fallback for misses.

The cache key must include all semantic inputs: source graph, specialization parameters, target, arithmetic flags, ABI/schema and relevant compiler version. Omitting a target feature or floating-point option can make an old binary appear valid under a new contract. Record cache hits, misses, compile latency, peak bytes and eviction behavior rather than measuring only warmed execution.

Keep code generation separate from permission to execute

The interpreter is the semantic reference and portable fallback. Constant folding must validate its input and preserve the unsigned arithmetic contract. Generated C++ should be inspected and tested against the interpreter for a bounded trusted corpus. Native compilation is an optional extension; no raw graph should cause the package to invoke a shell with user-controlled text.

A real JIT must additionally manage executable memory, code lifetime during concurrent use, cancellation, target capabilities and isolation. Writable/executable memory policies are one part of the design, not a complete sandbox. Process boundaries, capabilities, resource quotas and trusted compiler/runtime components may be needed depending on the threat model. The supplied code deliberately stops at restricted interpretation and source generation so it does not imply those protections have been implemented. [S16]

Worked engineering example

Eight binary material options allow 256 variants. At 80 KiB of code per variant, eagerly retaining all of them consumes 20,480 KiB, exactly 20 MiB, before metadata or other target variants. At 12 ms of compilation per variant, serial eager compilation adds 3.072 seconds. Specializing only the 20 common combinations initially reduces the immediate code budget to about 1.56 MiB and compilation to 240 ms, while uncommon cases use the fallback or a bounded later compilation path. Whether this improves the game depends on actual option frequencies and latency constraints.

Original worked check

Specializing four binary material options can create up to 2^4=16 variants before other parameters are considered. Uncontrolled variant growth can increase build time and runtime cache misses.

Practical assignment

Build a typed expression or material DSL with constant folding and a bounded interpreter or JIT route. Pass: invalid graphs fail safely, variant counts are measured and compiled outputs match the reference.

Full execution steps and acceptance checks are in LAB_WORKBOOK.pdf under this same lesson ID. Your warm-up file is `code/exercises/lesson_45_3.py`.

Reference starting point: code/gradlab/compiler.py: validate_expr, interpret, fold, cache_key, generate_cpp

Assigned study

Original assignment: R14: language implementation and JIT; R26: material and scripting concepts for comparison; Module 20: trust boundaries.

Focused expansion: R14 / S16: LLVM language and JIT tutorials; S13: generated-code semantics; R26: engine material and scripting interfaces. This package emits inspectable C++ but does not execute an untrusted native JIT.

Check your understanding

Attempt all three original questions before opening the key. A lesson also requires its reading, lab and graduate problems.

1. How many combinations do four independent binary options create?
2. Why limit the effects allowed in a DSL?
3. Why keep an interpreter alongside compiled execution?

Lesson 45.4 — Compiler/hardware co-design and graduate artifact review

Learning outcome: Connect a transformation to architecture and application evidence.

Planning: 22-34 hours plus overlapping Gate I integration. Includes assigned study and practical work; module/Gate I time overlaps.

Original teaching primer

Compiler decisions influence memory layout, instruction mix, register demand and parallel scheduling. Hardware features are useful only when software can express and exploit them efficiently. A co-design experiment changes one interface, representation or instruction opportunity and measures its complete effect. Avoid attributing a gain to hardware when it actually comes from a stronger software baseline. Graduate review requires a defensible claim, mathematical or semantic argument, implementation and reproducible evaluation. Gate I can focus on one earlier graduate topic, but must include baselines, ablations, uncertainty and limitations. Publication acceptance is not required; credible independent evaluation is. Preserve unsuccessful experiments and distinguish a reproduced known result from the learner's actual extension.

Explain the mechanism across the software/hardware boundary

A compiler transformation can change instruction count, data layout, register pressure, cache behavior and parallel occupancy at once. Attribute a claimed gain to evidence, not to the name of an instruction or architecture. A wider vector operation may require packing, unpacking, alignment handling and scalar tails. Saving arithmetic can increase memory traffic or reduce the number of resident workers.

Build a cost decomposition that links the generated code to the application's critical path. Compare a competent software baseline before proposing an accelerator or new instruction. An ablation should remove the proposed mechanism while keeping unrelated improvements fixed. A sensitivity study should vary a parameter that can expose the mechanism's limits, such as batch size, arithmetic intensity or register demand.

Complete one coherent research argument

Gate I requires a bounded reproduction plus a clearly identified extension, new setting or rigorous failure analysis. State exactly what prior result you are testing, which assumptions you retained and how your comparison differs. A reproduction discrepancy can become useful evidence when investigated carefully; it is not permission to invent a matching result or silently retune the baseline.

Deliver the derivation or semantic proof, code, workload definitions, raw data, analysis scripts, one ablation, one sensitivity study and a limitations section. Use equal-quality comparisons when methods change quality with speed. Identify the trial unit behind uncertainty estimates. Preserve negative cases and the tuning/evaluation split. This work prepares the thesis; it does not need to establish global novelty or publication acceptance.

The reviewer must be able to challenge the central claim

An independent reader should build or run the artifact and regenerate its main table from raw data. Ask the reader to inspect one failed case, alter one input and explain the legality or numerical assumptions behind the main transformation. Record discrepancies and revisions. A README that says results are reproducible is not evidence that another person actually reproduced them. [S14]

Even when Gate I focuses on rendering or simulation, this module still requires the small compiler optimization with semantic tests and generated-code inspection. The volume's breadth exercises support the specialization rather than disappearing once a favorite topic is chosen. Completion is recorded as demonstrated self-study work, with pending GPU, hardware or independent-review evidence marked honestly.

Worked engineering example

An engine frame takes 12 ms: 3 ms in a candidate kernel and 9 ms elsewhere. A proposed instruction halves the kernel arithmetic, apparently saving 1.5 ms. But the new layout adds 0.8 ms of packing and 0.9 ms of extra spill/transfer work. The new frame takes 9+1.5+0.8+0.9=12.2 ms, so speedup is 12/12.2, approximately 0.984. The kernel-only 2x number is real under its narrow boundary, but the integrated change is a regression. A sound report can conclude that the instruction is unattractive for this workload without treating the experiment as a failure of learning.

Original worked check

A new vector instruction that saves arithmetic but increases packing and spills may lose in the full engine. The experiment must measure both the generated kernel and its application integration.

Practical assignment

Complete Gate I and defend one compiler transformation's legality. Pass: an independent reader reruns the central experiment, the claimed extension is precisely identified and raw evidence supports the conclusions.

Full execution steps and acceptance checks are in LAB_WORKBOOK.pdf under this same lesson ID. Your warm-up file is `code/exercises/lesson_45_4.py`.

Reference starting point: code/run_experiments.py; code/native/main.cpp; templates/INDEPENDENT_REVIEW.md

Assigned study

Original assignment: R14 and R48: compilation and architecture; R49: co-design methodology; Module 41: research evaluation protocol.

Focused expansion: R14, R48 and R49: compiler/architecture interfaces and co-design; S14: artifact review principles. Gate I uses this curriculum's own rubric and does not award an ACM badge or an accredited degree.

Check your understanding

Attempt all three original questions before opening the key. A lesson also requires its reading, lab and graduate problems.

1. Why can a faster instruction fail to improve the application?
2. What distinguishes reproduction from extension?
3. Is publication acceptance mandatory for Gate I?

Module 45 project: compiler mini-project plus Gate I

The compiler requirement remains even when Gate I specializes elsewhere. Extend the prior tiny compiler or the supplied restricted reference with a verified IR subset and a dataflow/optimization pass. Cover loops and merges, state phi/alias/arithmetic semantics, and compare transformed execution against an interpreter. Inspect actual generated code and measure a bounded integration workload, not only expression evaluation in isolation.

Required artifacts: accepted language/IR contract, verifier checks, transfer functions or legality proof, original/transformed IR, directed and randomized tests, compiler flags, generated-code inspection and end-to-end measurements. Semantics 40 points, analysis/verifier 25, generated-code/performance evaluation 20, artifact/limitations 15. Suggested pass: 80/100 with no semantic mismatch. An existing constant-folding smoke test alone does not satisfy this project.

Complete Gate I using CAPSTONE_SPEC.pdf. The compiler mini-project and Gate I can overlap when compilation is the chosen depth area; otherwise they are distinct requirements within the volume's planning estimate. Independent review is required for Gate I, not silently replaced by the reference test suite.

Primary reading directory

R01-R55 retain the original program IDs. S01-S17 are focused additions for this expansion. Citations such as [S05] refer to the entries below; equations and worked cases not attributed to another source are original teaching derivations. External resources retain their own rights and are linked rather than redistributed. Moving documentation is not a promise of compatibility with an untested installed version. Record exact versions for your own artifact.

R11 — C++ Core Guidelines

C++ Core Guidelines editors

https://isocpp.github.io/CppCoreGuidelines/CppCoreGuidelines

Resource safety, interfaces, classes and concurrency; guidance rather than the normative language standard.

Source status: Inherited from the original program; not every moving page was revalidated for this expansion.

R14 — Kaleidoscope and LLVM tutorials

LLVM Project

https://llvm.org/docs/tutorial/

Compiler frontends, intermediate representation and JIT compilation.

Source status: Inherited from the original program; not every moving page was revalidated for this expansion.

R26 — Programming with C++ in Unreal Engine

Epic Games

https://dev.epicgames.com/documentation/en-us/unreal-engine/programming-with-cplusplus-in-unreal-engine

Engine comparison after C++ foundations, not a beginner substitute.

Source status: Inherited from the original program; not every moving page was revalidated for this expansion.

R37 — CUDA Programming Guide

NVIDIA

https://docs.nvidia.com/cuda/cuda-programming-guide/

GPU execution and memory; CUDA is an optional vendor-specific lab route.

Source status: Inherited from the original program; not every moving page was revalidated for this expansion.

R48 — 6.823 Computer System Architecture

MIT OpenCourseWare

https://ocw.mit.edu/courses/6-823-computer-system-architecture-fall-2005/

Graduate architecture foundations; historical course concepts supplemented with current specifications.

Source status: Inherited from the original program; not every moving page was revalidated for this expansion.

R49 — ECE5745 Complex Digital ASIC Design

Christopher Batten / Cornell University

https://www.csl.cornell.edu/courses/ece5745/

Graduate ASIC methodology, circuits, EDA and design-project expectations.

Source status: Inherited from the original program; not every moving page was revalidated for this expansion.

S13 — LLVM Language Reference Manual

LLVM Project

https://llvm.org/docs/LangRef.html

SSA, phi and arithmetic semantics. Record the LLVM version actually used in a learner experiment.

Source status: Focused primary source consulted September 28, 2026; see note for retrieval limits.

S14 — Artifact Review and Badging, version 1.1

ACM

https://www.acm.org/publications/policies/artifact-review-and-badging-current

Official search index and ACM terminology page verified; direct policy-page fetch returned 403. Supplement: https://www.acm.org/publications/badging-terms . This curriculum awards no ACM badge.

Source status: Focused primary source consulted September 28, 2026; see note for retrieval limits.

S15 — Auto-Vectorization in LLVM

LLVM Project

https://llvm.org/docs/Vectorizers.html

Loop/SLP vectorization, legality and diagnostics; primary compiler documentation.

Source status: Focused primary source consulted September 28, 2026; see note for retrieval limits.

S16 — LLVM Tutorial: Table of Contents

LLVM Project

https://llvm.org/docs/tutorial/

Frontend, IR and JIT tutorials. The bundled DSL stops at interpretation and source emission.

Source status: Focused primary source consulted September 28, 2026; see note for retrieval limits.